4332 CPU

Technical Reference Manual

Leonardo Palma

Version 1.0

May 2, 2026

Contents

1 Introduction **4**
- 1.1 Overview . . . 4
- 1.2 Main Features . . . 4
- 1.3 Limitations . . . 5
- 1.4 Implementation Estimates . . . 5
- 1.5 Structure of This Manual . . . 6

2 Instruction Set Architecture **7**
- 2.1 Data Formats . . . 7
- 2.2 Instruction Word Format . . . 7
  - 2.2.1 Conditional Execution (`CE`) . . . 8
  - 2.2.2 Data Width / Addressing Mode (`DW / AM`) . . . 8
- 2.3 Instruction Classes . . . 8
  - 2.3.1 A-type: Arithmetic and Data Instructions . . . 8
  - 2.3.2 F-type: Control Flow Instructions . . . 9
  - 2.3.3 S-type: System Instructions . . . 9
- 2.4 Registers . . . 9
  - 2.4.1 General-Purpose Registers . . . 9
  - 2.4.2 Special Registers . . . 10
  - 2.4.3 Condition Flags . . . 10
- 2.5 Extended Register Pairs . . . 10
- 2.6 Addressing . . . 11
- 2.7 Stack . . . 11
- 2.8 Calling Subroutines . . . 12

3 Interrupt and Exception System **13**
- 3.1 Overview . . . 13
- 3.2 Interrupt Vectors and Priority . . . 13
- 3.3 Interrupt Controller Register (RIC) . . . 13
- 3.4 Interrupt Vector Table (RIV) . . . 14
- 3.5 Hardware Interrupt Entry Sequence . . . 14
- 3.6 Returning from Interrupts (IRET) . . . 14
- 3.7 Software Interrupts . . . 15
- 3.8 Privilege Faults . . . 16
- 3.9 Illegal Opcode Faults . . . 16

4 Memory Organisation **17**

4.1 Address Space . . . 17
4.2 Instruction Bus . . . 17
4.3 Data Bus . . . 17
4.4 Memory-Mapped Peripherals . . . 18

5 Hardware Architecture 19
5.1 Pipeline Overview . . . 19
5.2 Micro-Sequencer . . . 19
5.3 Hazard Handling . . . 19
5.3.1 RAW Hazards . . . 19
5.3.2 Branch Hazards . . . 20
5.3.3 Interrupt Hazards . . . 20
5.4 LOADIMM and 2-Word Instructions . . . 20
5.5 PC Management . . . 20
5.6 Control Word ROM . . . 21

6 Integration and Synthesis 22
6.1 Top-Level Ports . . . 22
6.2 Clock and Reset . . . 22
6.3 Connecting to the Memory Bus . . . 23
6.4 Interrupt Wiring . . . 23
6.5 Synthesis Guidelines . . . 23
6.5.1 BRAM Inference . . . 23
6.5.2 Timing . . . 23
6.5.3 Optimising for Area . . . 24

7 Simulation 25
7.1 Testbench Overview . . . 25
7.2 Test Suites . . . 25
7.3 Build System . . . 25
7.3.1 Running Simulation . . . 26
7.3.2 Synthesis . . . 26
7.4 Requirements . . . 26

8 Assembler 27
8.1 Overview . . . 27
8.2 Usage . . . 27
8.3 Register Syntax . . . 27
8.4 Mnemonics and Suffixes . . . 27
8.5 Addressing Modes for J and CALL . . . 28
8.6 LOADIMM . . . 28
8.7 Directives . . . 28
8.8 Labels . . . 28
8.9 Number Literals . . . 29
8.10 Comments . . . 29

A Instruction Set Reference 30
A.1 Instruction Listing by Group . . . 30
A.2 Instruction Descriptions . . . 31

B Assembler Language Reference **34**
B.1 Source File Format 34
B.2 Comments 34
B.3 Labels 34
B.4 Number Literals 35
B.5 String Escape Sequences 35

C Control Word Layout **36**

D Register File **40**

Chapter 1

Introduction

1.1 Overview

The 4332 is a 16-bit microcoded CPU designed for FPGA implementation on the Xilinx Artix-7 (Basys 3 development board). The name encodes its key parameters: $2^4 = 16$-bit data width, $2^3 = 8$ general-purpose registers, and a 32-bit control word — hence *4332*.

At its core the 4332 is a NISC (*No Instruction Set Computer*). Every opcode is decoded into a 32-bit control word (CW), looked up from a 512-entry ROM indexed by {`opcode[5:0], step[2:0]`}. This means adding a new instruction is as simple as writing a new CW entry: no changes to the datapath, no changes to the decode logic. The same property makes the 4332 unusually hackable — it is, in principle, possible to make it decode a completely different ISA by replacing the CW ROM.

Despite its simplicity the 4332 does not compromise on system features. It has a proper privilege architecture, a vectored interrupt controller with 8 vectors, and a 3-stage pipeline — all in a footprint of under 1100 LUTs on an Artix-7.

1.2 Main Features

- **Microcoded NISC decode.** Every instruction maps to a 32-bit control word looked up from a 512-entry BRAM-inferred ROM. The CW directly drives every datapath signal. Adding a new opcode is a matter of writing a new entry in the ROM initialisation function; no RTL changes are required.
- **3-stage pipeline.** FETCH, DECODE, and EXECUTE stages. RAW hazards are resolved by a one-cycle stall with bubble injection. Branches flush the pipeline on taken, producing a single-cycle penalty bubble. There is a static branch predictor, as it was cheap to implement and gives noticeable performance gains.
- **Privilege architecture.** Hardware user/kernel mode separation. Any attempt to execute a privileged instruction from user mode raises a *privilege fault* (vector 6), which the interrupt controller services before the offending instruction can have any effect on architectural state.
- **Vectored interrupt system.** 8 interrupt vectors with fixed priority (NMI >

PRIV_FAULT > ILL_OPCODE > SOFT_INT > IRQ3–IRQ0). The hardware entry sequence automatically saves the PC, privilege mode, faulting instruction, and faulting memory address into dedicated exception registers (`XR0`–`XR3`) — no software prologue is required to diagnose a fault.

- **Suffix-based conditionality.** Any instruction can be made conditional by appending a condition suffix (`.z`, `.c`, or `.n`). When the condition is not met, the instruction executes as a NOP: no writeback, no memory access, no PC redirect, no side effects — but the micro-sequencer still runs to completion so that multi-word instructions drain the pipeline correctly.
- **Variable-width ALU.** The `DW` field in the instruction word selects 16-bit, 8-bit low, 8-bit high, or 32-bit (extended register pair) operations for ALU instructions. The same hardware handles all four widths.
- **Customisable decode ROM.** Because every instruction is just a CW entry, extending the ISA requires no datapath modification. The `make_cw` helper function with named parameters keeps opcode definitions readable and maintainable.
- **Small footprint.** Under 1100 LUTs and approximately 520 flip-flops on an Artix-7, occupying about 5% of the LUT budget of a Basys 3 board. Plenty of headroom for a full SoC around the CPU.

1.3 Limitations

The 4332 has a 16-bit address space, giving a maximum of 65,536 addressable word locations (byte addressing, so 32,768 word-aligned words). There is no memory management unit, no virtual memory, and no 32-bit memory addressing. The `.d` (double-word) width suffix is meaningful only for register-to-register ALU operations on extended register pairs (Section 2.5); it is not supported for memory access instructions. The data bus is 16 bits wide; 32-bit memory transfers must be implemented as two consecutive 16-bit loads or stores.

Interrupt nesting is not implemented by design. Once the interrupt controller begins servicing a vector, GIE is cleared and further interrupts are deferred until `IRET` re-enables them. This simplifies debugging and interrupt handler code significantly.

The CPU currently boots into kernel mode and does not have a formal mechanism for transitioning to user mode other than `SETMODE`.

About 300 LUTs are used to create the asynchronous CW ROM. A future version featuring a four stage pipeline and higher clock speeds is planned. This specific version can run at 100MHz, the highest clock my board can run.

1.4 Implementation Estimates

Typical synthesis results for the 4332 CPU on the Artix-7 (Basys 3 board) are shown in Table 1.1. These figures are approximate and depend on tool version, constraint setup, and design utilization.

Table 1.1: 4332 CPU resource footprint (Artix-7 XC7A35T)

Resource	**Estimate**
LUTs	1061 (5.25% of XC7A35T)
Flip-flops	524 (1.26%)
BRAM	1
Clock	$100MHz$

1.5 Structure of This Manual

A general description of the 4332 architecture from a software developer's point of view is found in Chapter 2, *Instruction Set Architecture*. Pipeline and hazard handling from a hardware perspective is covered in Chapter 5, *Hardware Architecture*. Chapter 3, *Interrupt and Exception System*, documents the interrupt controller, vector table, and entry/return sequences in detail. Memory organisation and the bus interface are described in Chapter 4, *Memory Organisation*. Integration with the rest of a Basys 3 SoC design — ports, synthesis, and timing — is the subject of Chapter 6, *Integration and Synthesis*. Chapter 7, *Simulation*, covers the testbench and build system.

Appendices contain the full instruction set reference (Appendix A), assembler syntax (Appendix B), control word layout (Appendix C), and the register file (Appendix D).

Chapter 2

Instruction Set Architecture

2.1 Data Formats

The 4332 is a 16-bit CPU. All general-purpose registers hold 16-bit values. Signed values are encoded in two's complement. The ALU sign-extends 8-bit results to 16 bits when writing back through the W8 path. Memory is byte-addressed; word accesses are always to even addresses (the least significant bit of the address is forced to zero by hardware).

Extended register operations (`.d`) treat pairs of adjacent registers as a single 32-bit value, with the even-numbered register holding the low word and the odd-numbered register holding the high word.

2.2 Instruction Word Format

Every 4332 instruction occupies exactly one 16-bit word in the instruction stream, with the exception of `LOADIMM` and 2-word addressing modes for `J/CALL`, which are followed by a 16-bit extension word. The base instruction word is laid out as follows:

[15:10]	**[9:7]**	**[6:4]**	**[3:2]**	**[1:0]**
OPCODE (6 bits)	RD (3 bits)	RS (3 bits)	DW / AM (2 bits)	CE (2 bits)

`OPCODE [15:10]` 6-bit instruction opcode. Selects the CW ROM entry together with the 3-bit micro-step counter. See Appendix A for the full opcode table.

`RD [9:7]` Destination register selector. Indexes into `%0`–`%7`. For instructions that only have a source operand this field is ignored by the datapath (though it is still encoded as zero by the assembler).

`RS [6:4]` Source register selector. Indexes into `%0`–`%7`. Also used as the vector index for `READRIV/WRITERIV`, and as the indirect jump target register for `J/CALL`.

`DW / AM [3:2]` Dual-purpose field. Its interpretation depends on the instruction class (see Section 2.2.2). For arithmetic and memory instructions it is the data width field (`DW`); for control-flow instructions it is the addressing mode field (`AM`).

`CE [1:0]` Conditional execution field. Applies to every instruction class. When the condition is not met the instruction executes as a NOP: no writeback, no memory access, no PC redirect, no side effects. The micro-sequencer still runs to completion so that multi-word instructions drain the pipeline correctly and the instruction stream pointer is left in the correct state.

2.2.1 Conditional Execution (CE)

Table 2.1: Conditional execution field encoding

CE[1:0]	Suffix	Condition
00	(none)	Always — unconditional execution
01	.z	Execute if Zero flag is set (Z=1)
10	.c	Execute if Carry flag is set (C=1)
11	.n	Execute if Negative flag is set (N=1)

Any instruction can carry a condition suffix. For example, `add.z %1, %2` adds `%2` into `%1` (via `%0`) only if the Zero flag is set; `j.c loop` jumps only if Carry is set. There is no dedicated conditional-branch opcode family.

2.2.2 Data Width / Addressing Mode (DW / AM)

Bits `[3:2]` serve two distinct roles depending on the instruction class.

Data instructions (width field, DW). For arithmetic, logic, and memory instructions the `DW` field selects the width of the operation. This is only used by the datapath when the CW has `WIDTH_FROM_INST` set. Otherwise the CW supplies the width directly, which is how fixed-width system instructions like `LOADIMM` work.

Table 2.2: Data width field encoding

DW[1:0]	Suffix	Meaning
00	.w	16-bit word operation (default; suffix is optional)
01	.l	8-bit low byte (bits 7:0 of the register)
10	.h	8-bit high byte (bits 15:8 of the register)
11	.d	32-bit double-word via extended register pair

Control-flow instructions (addressing mode, AM). For `J` and `CALL`, the same two bits encode the addressing mode. The mode is chosen automatically by the assembler based on operand form; no explicit suffix is needed.

2.3 Instruction Classes

2.3.1 A-type: Arithmetic and Data Instructions

A-type instructions are the workhorse class. They cover all ALU operations, memory access, and stack operations. Bits `[3:2]` are the `DW` (data width) field.

Table 2.3: Addressing mode field encoding (J / CALL)

AM[1:0]	Assembler form	Description
00	`j label`	Direct: absolute target in extension word (2-word instruction)
01	`j %rs`	Indirect: target is `%RS` (1-word instruction)
10	`j %rs, offset`	Indirect+offset: target is `%RS` + extension word (2-word)
11	—	Illegal addressing mode — triggers illegal-opcode fault (vector 5)

Most A-type instructions are one word. `LOADIMM` is the exception: it is always two words, consuming the following instruction word as a 16-bit immediate. The `DW` and `CE` fields in the `LOADIMM` opcode word are both ignored; the `RD` field selects the destination register.

2.3.2 F-type: Control Flow Instructions

F-type instructions include `J`, `CALL`, `RET`, and `IRET`. Bits `[3:2]` are the `AM` (addressing mode) field. Bits `[1:0]` remain the `CE` conditional execution field.

Direct and indirect-with-offset modes produce a two-word instruction; indirect mode produces a one-word instruction. Examples:

```
j      loop        ; Direct: 2 words, absolute target
j      %3          ; Indirect: 1 word, target = %3
j      %3, 0x10    ; Indirect+offset: 2 words, target = %3 + 0x10
j.z    done        ; Conditional direct jump: taken only if Z=1
j.c    %2          ; Conditional indirect: taken only if C=1
```

2.3.3 S-type: System Instructions

System instructions (opcodes 0x30–0x39) require kernel mode to execute. Attempting any system instruction from user mode raises a privilege fault before the instruction has any effect. These instructions manipulate the interrupt controller (`ENIRQ`, `DISIRQ`, `WRITERIC`), read/write interrupt vectors (`READRIV`, `WRITERIV`), and query or change privilege mode (`GETMODE`, `SETMODE`).

2.4 Registers

2.4.1 General-Purpose Registers

The 4332 has eight 16-bit general-purpose registers, denoted `%0`–`%7`. All registers are zero-initialised on reset. From the hardware's perspective they are all symmetric: any register can be used for any purpose.

By convention `%0` is the implicit accumulator for most ALU instructions (instructions that do not set `RD_FROM_INST` always write back to `%0`). The assembler enforces even

register numbers for `.d` (double-word) operations, since the upper half of a 32-bit result is written to `%RD+1`.

2.4.2 Special Registers

Special registers are not directly addressable by general ALU instructions. They are accessed via dedicated system instructions or implicitly by hardware during pipeline events.

Table 2.4: 4332 special-purpose registers

Register	Reset Value	Description
`RSP`	`0xBFFE`	Stack pointer. Initialised to the top of RAM bank 0.
`RPC`	`0x0000`	Program counter.
`RIP`	`0x0000`	Saved PC captured on interrupt entry (restored by `IRET`).
`XR0`	`0x0000`	Faulting PC (address of the instruction that caused the fault).
`XR1`	`0x0000`	Faulting vector number (0–7).
`XR2`	`0x0000`	Faulting instruction word.
`XR3`	`0x0000`	Faulting memory address.
`RM`	`0x0001`	Mode register. Bit 0 = current mode (1=kernel, 0=user). Bit 1 = previous mode.
`RIC`	`0x0000`	Interrupt controller register (GIE, enable mask, pending).
`RIV[0..7]`	`0x0000`	Interrupt vector table. 8 entries, one per vector.

2.4.3 Condition Flags

The 4332 has four condition flags updated by ALU operations:

Table 2.5: Condition flags

Flag	Name	Set when...
Z	Zero	The result of the operation is zero (within the active width).
C	Carry	An unsigned carry or borrow occurred. For shifts, this captures the shifted-out bit.
N	Negative	The most significant bit of the result is 1 (within the active width).
V	Overflow	Signed overflow occurred (add/subtract only).

The `PASSA` and `PASSB` ALU operations do not update flags. All other ALU operations (including `CMP`, which performs subtraction without writeback) do update all four flags. The width used for flag evaluation matches the effective width of the instruction (`eff_width`).

2.5 Extended Register Pairs

The eight general-purpose registers may be paired into four 32-bit *extended registers* (ER):

Table 2.6: Extended register pairs

ER	Registers	Layout
ER0	%0, %1	%0 = low word, %1 = high word
ER1	%2, %3	%2 = low word, %3 = high word
ER2	%4, %5	%4 = low word, %5 = high word
ER3	%6, %7	%6 = low word, %7 = high word

When the `.d` suffix is used on an ALU instruction, the result is written as a 32-bit value: the low 16 bits go to `RD` and the high 16 bits go to `RD+1`. The destination register should always be even when using `.d`; the assembler does not currently enforce this, but the hardware will silently produce incorrect results for odd-register destinations.

Loading a 32-bit immediate requires two `LOADIMM` instructions:

```
loadimm %0, 0xBEEF    ; low word  -> %0
loadimm %1, 0xDEAD    ; high word -> %1
; ER0 now holds 0xDEADBEEF
```

2.6 Addressing

The 4332 has a 16-bit address space. Addresses are byte addresses; word-aligned accesses use even addresses (the low bit is ignored for word reads/writes). The address space is divided into four 16 KB banks:

Table 2.7: Address space layout

Range	Size	Region
0x0000–0x3FFF	16 KB	ROM (writable after startup via memory bus)
0x4000–0x7FFF	16 KB	Peripherals
0x8000–0xBFFF	16 KB	RAM bank 0 (stack default here)
0xC000–0xFFFF	16 KB	RAM bank 1

All memory addressing in ALU instructions is indirect: the memory address must be in a register (`%RS` for loads, `%RD` for stores). There is no instruction that directly encodes a memory address in the instruction word; to access an arbitrary address, load it into a register with `LOADIMM` first.

2.7 Stack

The stack pointer is stored in the special register `RSP`. On reset it is initialised to `0xBFFE`, which is the top of RAM bank 0. The stack grows downward. Push decrements `RSP` before writing; pop reads at the current `RSP` before incrementing.

The hardware `PUSH` and `POP` instructions operate implicitly on `RSP`. There is no way to use a general register as an alternative stack pointer through these instructions; if an alternate stack is needed it must be implemented manually with `LD`/`ST` and arithmetic.

```
; Manual push of %2 to an alternate stack in %5
dec    %5           ; RSP-- equivalent
st     %5, %2       ; mem[%5] <- %2

; Manual pop from alternate stack
ld     %2, %5       ; %2 <- mem[%5]
inc    %5           ; RSP++ equivalent
```

Before using the stack in any interrupt handler or subroutine, the `RSP` register must already point to valid RAM. Since `RSP` resets to the top of RAM bank 0, simple programs can use the stack immediately after reset without any initialisation.

2.8 Calling Subroutines

Subroutine calls are implemented by the `CALL` instruction, which pushes the return address onto the hardware stack (via `RSP`) and then jumps to the target. It supports the same three addressing modes as `J`:

```
call   proc_addr      ; Direct: push ret addr, jump to immediate target
call   %3             ; Indirect: push ret addr, jump to %3
call   %3, 0x10       ; Indirect+offset: push ret addr, jump to %3+0x10
```

The return address pushed onto the stack is the address of the word immediately following the `CALL` instruction (or its extension word, for 2-word forms). Returning from a subroutine is done with `RET`:

```
ret                   ; RPC <- pop()
```

For nested calls, the caller's return address is naturally preserved on the stack, so no special save/restore is needed in the calling convention. The stack discipline handles arbitrary call depth as long as sufficient stack space is available.

A typical calling convention for 4332 programs is: pass arguments in `%1`–`%6`, with `%0` used as the scratch/accumulator register for results. The callee is responsible for preserving any registers it uses that the caller might depend on (callee-saved convention). Since the ISA has only 8 registers, tight code may choose to pass arguments on the stack for functions with many parameters.

Chapter 3

Interrupt and Exception System

3.1 Overview

The 4332 interrupt system has 8 vectors with fixed priority. There are two categories: *synchronous exceptions*, which are generated by the CPU itself in response to instruction execution, and *asynchronous interrupts*, which arrive from external logic or software.

Interrupt nesting is not supported. When interrupt entry begins, GIE is cleared immediately, deferring any further interrupts until `IRET` is executed.

3.2 Interrupt Vectors and Priority

Table 3.1: Interrupt vector table

Vector	Priority	Type	Name	Source
0	4 (lowest)	Async	IRQ0	External hardware interrupt line 0
1	3	Async	IRQ1	External hardware interrupt line 1
2	2	Async	IRQ2	External hardware interrupt line 2
3	1	Async	IRQ3	External hardware interrupt line 3
4	0	Sync	SOFT_INT	Software interrupt (`INT` instruction)
5	—	Sync	ILL_OPCODE	Illegal opcode detected in DECODE stage
6	—	Sync	PRIV_FAULT	Privileged instruction in user mode
7	7 (highest)	Async	NMI	Non-maskable interrupt

Synchronous exceptions (vectors 5 and 6) beat asynchronous IRQs. Within the async group, NMI (vector 7) always fires regardless of GIE. Among the maskable hardware IRQs (0–3), vector 0 has the highest priority and vector 3 the lowest.

3.3 Interrupt Controller Register (RIC)

The interrupt controller state is accessible via the `RIC` register, read with `READRIC` and written with `WRITERIC`. The layout is:

Table 3.2: RIC register layout

Bits	Name	Description
[15:8]	PENDING[7:0]	Interrupt pending bits. Set when an interrupt is latched, cleared by hardware during entry.
[7:1]	ENABLE[7:1]	Per-vector enable bits for vectors 7 down to 1.
[0]	GIE	Global Interrupt Enable. 0 on reset, set by `ENIRQ`, cleared by `DISIRQ` or interrupt entry.

Vectors 0–3 (hardware IRQs) are only serviced when both the individual enable bit (in `RIC`) and GIE are set. Vector 7 (NMI) fires unconditionally regardless of GIE. Vectors 5 and 6 (ILL_OPCODE and PRIV_FAULT) are synchronous and always serviced when they occur.

3.4 Interrupt Vector Table (RIV)

The 4332 maintains an 8-entry interrupt vector table in registers `RIV[0]`–`RIV[7]`. Each entry holds the 16-bit address of the interrupt handler for that vector. Entries are written with `WRITERIV` and read with `READRIV`:

```
; Set handler for IRQ0 (vector 0) to address stored in %0
writeriv  %0, %1    ; RIV[%1] <- %0  (RS field is vector index)

; Typical setup sequence
loadimm   %0, irq0_handler
loadimm   %1, 0             ; vector 0
writeriv  %0, %1
```

All vector registers initialise to `0x0000` on reset. If an interrupt fires before its vector has been written, execution jumps to address 0, which typically contains the reset code. This is almost certainly not what you want, so vectors should always be initialised before enabling interrupts.

3.5 Hardware Interrupt Entry Sequence

When the interrupt controller detects a pending and enabled interrupt at an instruction boundary (`LAST_STEP` = 1), it begins a 5-step hardware entry micro-sequence. The CPU is stalled for these 5 cycles; no instructions from the normal stream are fetched or executed during this time.

The pipeline is flushed on step 4, which means any instruction already in the FETCH or DECODE stage at the time of the interrupt is squashed. The interrupt is always taken at a clean instruction boundary.

3.6 Returning from Interrupts (IRET)

The `IRET` instruction restores the CPU state saved during interrupt entry:

Table 3.3: Interrupt entry micro-sequence

Step	Action	Notes
0	GIE ← 0; clear pending bit	Disables further interrupt processing.
1	RIP ← RPC	Saves the next-PC (the instruction that would have executed).
2	RM_PREV ← RM; RM ← kernel	Saves privilege mode, switches to kernel.
3	XR0–XR3 ← context	Saves faulting PC, vector, instruction word, and memory address.
4	RPC ← RIV[vec]; flush	Jumps to handler; pipeline is flushed.

1. `RPC` ← `RIP` (restores saved program counter).
2. `RM` ← `RM_PREV` (restores previous privilege mode).
3. `GIE` ← 1 (re-enables interrupts).
4. Pipeline is flushed; execution resumes from the saved PC.

`IRET` is a kernel-only instruction (it has `PRIV_CHECK` set in its CW). Attempting it from user mode raises a privilege fault, which is probably not what you want inside an exception handler.

A minimal interrupt handler structure:

```
irq0_handler:
  ; Save any registers you need to clobber
  push  %0
  push  %1

  ; ... handle the interrupt ...

  ; Restore and return
  pop   %1
  pop   %0
  iret
```

Since GIE is cleared on entry and restored by `IRET`, no explicit `DISIRQ`/`ENIRQ` is needed in a simple handler.

3.7 Software Interrupts

The `INT` instruction raises software interrupt vector 4 (SOFT_INT). It sets the pending bit for vector 4 in `RIC`; the interrupt is then processed at the next instruction boundary if GIE is set and the enable bit for vector 4 is set. `INT` can be used to implement system calls from user mode.

3.8 Privilege Faults

A privilege fault (vector 6) is raised any time a system instruction is executed while the CPU is in user mode (`RM` bit 0 = 0). The fault is detected during the EXECUTE stage, before the instruction has any architectural effect — no state change occurs. The exception context registers capture the faulting address and instruction word, allowing the kernel to diagnose the fault.

3.9 Illegal Opcode Faults

An illegal opcode fault (vector 5) is raised when the DECODE stage encounters an opcode whose CW ROM entry is the NOP control word (i.e. an opcode with no defined action, other than the actual `NOP` instruction). It is also raised when a `J` or `CALL` instruction uses the reserved addressing mode `AM=11`. The fault fires before the instruction reaches EXECUTE.

Chapter 4

Memory Organisation

4.1 Address Space

The 4332 has a 16-bit address space divided into four 16 KB banks as described in Table 4.1. All addresses are byte addresses; word-aligned operations address even bytes and the least significant address bit is ignored.

Table 4.1: Address space layout

Start	End	Size	Description
`0x0000`	`0x3FFF`	16 KB	ROM / Program memory
`0x4000`	`0x7FFF`	16 KB	Peripherals
`0x8000`	`0xBFFF`	16 KB	RAM bank 0 (default stack)
`0xC000`	`0xFFFF`	16 KB	RAM bank 1

4.2 Instruction Bus

The CPU presents a 16-bit `iaddr` output (the program counter value) and receives 16-bit `idata` as the next instruction. In the testbench and top-level design, the instruction bus is connected to a synchronous ROM. The instruction address is registered at the end of each cycle; the ROM must present valid data on the next clock edge. This is the "Option C" fetch/execute overlap strategy that handles the 1-cycle read latency of synchronous BRAM without stalling.

The instruction bus is read-only; there is no instruction cache and no write path.

4.3 Data Bus

The data bus is a simple single-cycle interface with the following signals:

`dwe` and `dbe` are gated by `cond_met`: a condition-not-met store or load does not produce any bus transaction.

Table 4.2: Data bus signals

Signal	Dir	Width	Description
`daddr`	out	16	Memory address
`ddata_w`	out	16	Write data
`ddata_r`	in	16	Read data
`dwe`	out	1	Write enable (synchronous write on rising edge)
`dbe`	out	1	Bus enable (asserted for memory reads)
`dwmask`	out	2	Byte write mask: bit 0 enables low byte, bit 1 enables high byte

The byte write mask (`dwmask`) follows the convention that a *0* in the mask means the corresponding byte is written; a *1* means it is held. This matches the active-low enable convention used in the RAM module. For word operations both bits are 0; for byte low (`.l`) bit 1 is 1; for byte high (`.h`) bit 0 is 1.

4.4 Memory-Mapped Peripherals

Peripheral access in the `0x4000`–`0x7FFF` range is routed to the `peripherals` entity by the `memory_bus` module. The peripheral address is the low byte of the memory address (`address[7:0]`). The following peripheral registers are defined:

Table 4.3: Peripheral register map (peripheral address)

Address	R/W	Description
`0x00`	R	Button inputs: `data[5:0]` = {button_0, btn[4:0]}
`0x00`	W	IO port register: `data[3:0]` = {ioport_3, ioport_2, ioport_1, ioport_0}
`0x01`	R	Switch inputs: `data[15:0]` = sw[15:0]
`0x02`	W	LED outputs: `data[15:0]` = led[15:0]
`0x03`	W	7-segment segments: `data[6:0]` = seg[6:0]
`0x04`	W	7-segment anode select: `data[3:0]` = an[3:0]
`0x05`	R/W	SPI data: write to transmit, read for received byte
`0x06`	R	SPI status: `data[0]` = busy

Chapter 5

Hardware Architecture

5.1 Pipeline Overview

The 4332 uses a 3-stage pipeline:

- **FETCH** — Presents the program counter to the instruction ROM and latches the returned instruction word into the FD (Fetch/Decode) pipeline register.
- **DECODE** — Reads the CW ROM, looks up the register file for operand values, and latches results into the DE (Decode/Execute) pipeline register.
- **EXECUTE** — Drives the ALU, memory bus, flag registers, PC, stack pointer, and register file writeback based on the current control word.

Each stage takes exactly one clock cycle for single-step instructions. Multi-step instructions (those with `LAST_STEP` = 0 in the first CW entry) stall the FETCH stage and keep the instruction in EXECUTE, advancing the micro-step counter and selecting successive CW ROM entries on each subsequent cycle.

5.2 Micro-Sequencer

The micro-sequencer is a 3-bit step counter. On each rising edge where `LAST_STEP` = 1, it resets to 0. Otherwise it increments. The CW ROM is indexed by {`opcode[5:0]`, `step[2:0]`}, giving up to 8 micro-steps per opcode. The FETCH stage is suppressed while `micro_step` $\neq$ `0`, ensuring that the instruction currently in EXECUTE holds the pipeline until it completes.

5.3 Hazard Handling

5.3.1 RAW Hazards

Read-After-Write (RAW) hazards occur when an instruction in DECODE reads a register that the instruction currently in EXECUTE is writing. The 4332 does not use forwarding; instead, it inserts a one-cycle stall:

1. A bubble (NOP control word) is injected into the DE register.

2. The FD register is frozen (FETCH does not advance).
3. On the next cycle, the EXECUTE stage has completed its writeback and the correct register value is available.

The stall condition is: EXECUTE has `REG_WE` = 1, `cond_met` = 1, and the DECODE instruction's RS or RD index matches the EXECUTE writeback destination.

5.3.2 Branch Hazards

When a branch is taken, the pipeline is flushed: the FD and DE registers are invalidated (their `valid` bits cleared and their CW fields set to `CW_NOP`). This produces a single-bubble penalty on every taken branch. Not-taken branches (condition not met) do not flush the pipeline.

For 2-word jump instructions, the extension word is consumed in step 0 of the EXECUTE stage. The fetch stage advances past the extension word regardless of whether the branch is taken, ensuring the instruction following the extension word is the next to be fetched.

5.3.3 Interrupt Hazards

Interrupts are taken only at instruction boundaries (`LAST_STEP` = 1). When an interrupt is taken, the pipeline is flushed before the hardware entry sequence begins. This ensures a clean architectural state is captured in the exception registers.

5.4 LOADIMM and 2-Word Instructions

`LOADIMM` is the only data instruction that uses an extension word. In step 0 of EXECUTE, the `fd_instr` register (which holds the word following the opcode) is captured into the `imm_word` latch, and `RPC` is incremented to skip past it. In step 1, `WB_IMM_WORD` sources the writeback data from `imm_word` and the register is written.

The FETCH stage is suppressed during step 0, so `fd_instr` holds the extension word stably throughout step 0 without being overwritten by a new fetch.

5.5 PC Management

The program counter `RPC` is maintained by the EXECUTE stage. It is incremented in the FETCH stage as instructions are consumed, and may be redirected by:

- A taken branch or jump (`PC_JUMP`).
- A subroutine call (`PC_JUMP` with stack push).
- A return instruction (`PC_RET`): RPC ← `ddata_r` (popped from stack).
- An interrupt entry (`PC_INT` or hardware micro-sequence step 4): RPC ← `RIV[vec]`.
- An IRET (`PC_IRET`): RPC ← `RIP`.
- `HLT` (`PC_STALL`): RPC is held at the current instruction address.

5.6 Control Word ROM

The CW ROM is a 512-entry, 32-bit wide array implemented in VHDL as a constant initialised by the `init_cw_rom` elaboration-time function. Vivado infers this as a BRAM. The index is {`opcode[5:0], step[2:0]`}.

New instructions can be added by writing a new entry block in `init_cw_rom` using the `make_cw` helper, which accepts named parameters for every CW field and defaults all others to safe (NOP-like) values. This is the entire extent of RTL change required to add an instruction.

Chapter 6

Integration and Synthesis

6.1 Top-Level Ports

The `cpu` entity has the following ports:

Table 6.1: CPU port list

Port	Dir	Width	Description
`clk`	in	1	System clock (rising edge triggered)
`reset`	in	1	Synchronous active-high reset
`iaddr`	out	16	Instruction address (program counter)
`idata`	in	16	Instruction data from ROM
`daddr`	out	16	Data bus address
`ddata_w`	out	16	Data bus write data
`ddata_r`	in	16	Data bus read data
`dwe`	out	1	Data bus write enable
`dbe`	out	1	Data bus bus enable (read)
`dwmask`	out	2	Byte write mask
`irq`	in	4	External interrupt requests (active high, level)

6.2 Clock and Reset

All flip-flops in the CPU are triggered on the rising edge of `clk`. The `reset` signal is synchronous and active-high. If the board reset comes from an asynchronous source (e.g. a debounced button), a two-flip-flop synchronisation chain should be added before connecting it to the CPU.

On an SRAM-based FPGA like the Artix-7, flip-flops are initialised to known states when the bitstream is loaded, so in principle the CPU can run without a reset signal. In practice the reset is useful during simulation and debugging.

6.3 Connecting to the Memory Bus

The CPU's instruction bus is designed to connect directly to a synchronous ROM (BRAM or initialised distributed RAM). The CPU presents an address on `iaddr` and expects the data to be latched into `idata` on the following rising edge. This means the ROM must have a one-cycle read latency, which is the natural behaviour of BRAM on the Artix-7.

The `memory_bus` entity handles routing of the data bus to ROM, peripherals, and RAM. It decodes the top two address bits to select the target and drives the appropriate enable and write signals.

6.4 Interrupt Wiring

The four external IRQ lines (`irq[3:0]`) are active-high, level-sensitive signals from the CPU's perspective. The interrupt controller samples them every cycle; they must be synchronous with `clk`. If your interrupt source is asynchronous (e.g. an external pin), add a two-stage synchronisation chain clocked by `clk` before connecting to `irq`.

Note that the individual enable bits for IRQ0–IRQ3 in `RIC` must be set (along with GIE) before any hardware IRQ can be taken. The IRQ lines should be held low until the firmware has set up interrupt vectors and enabled the relevant channels.

6.5 Synthesis Guidelines

6.5.1 BRAM Inference

The CW ROM is initialised by the `init_cw_rom` function at elaboration time. Vivado will infer this as a block RAM automatically. If the inference fails (visible as the CW ROM appearing in LUT usage rather than BRAM usage in the synthesis report), the most likely cause is that the array is too small — the 512-entry, 32-bit wide array is above the threshold for BRAM inference on most Vivado versions.

Both the RAM banks and the ROM use the same synchronous read style, which is BRAM-friendly. Do not change these to asynchronous reads without also adjusting the fetch/execute pipeline timing.

6.5.2 Timing

The natural critical path runs from the register file output through the ALU and back to the register file input (RAW path through the scratchpad in the EXECUTE stage). A secondary critical path runs from the CW ROM output through the datapath muxes. With default Vivado settings the design should meet timing at 50–100 MHz on the Artix-7; the Basys 3 runs at 100 MHz.

If timing fails, the most common remedies are:

1. Ensure the CW ROM is inferred as BRAM (avoids LUT-path delays on the CW bus).
2. Pipeline the ALU operand mux if the multiplexer depth becomes the critical path.

3. Add a register stage between the CW ROM output and the datapath (this adds one pipeline stage and requires adjusting the micro-sequencer).

6.5.3 Optimising for Area

The biggest consumer of LUTs is the ALU and operand mux. The 32-bit wide ALU path (for `.d` operations) is the main contributor; if double-word operations are not required the 32-bit paths can be trimmed to 16 bits, cutting approximately 20–30% of the LUT usage.

Chapter 7

Simulation

7.1 Testbench Overview

The 4332 package includes a self-checking testbench (`cpu_tb.vhd`) that verifies CPU functional correctness across 13 test suites. The testbench instantiates the CPU, provides a simple instruction ROM (an array loaded by the test process), and a simple data RAM modelled as a protected shared variable.

7.2 Test Suites

Table 7.1: Testbench test suites

Suite	Name	What it tests
1	Reset	PC is zero during and immediately after reset.
2	NOP pipeline	PC advances correctly through three sequential NOPs.
3	LOADIMM	Two-word load of a 16-bit immediate into a register.
4	MOV	Register-to-register move with explicit destination.
5	ALU	ADD, SUB, AND, OR, XOR, SHL, SHR, INC, DEC.
6	CMP + branches	CMP sets flags; JZ taken when Z=1; JNZ not-taken when Z=1.
7	JMP indirect	1-word indirect jump via register.
8	CALL / RET	1-word indirect call; callee sets register; RET returns.
9	LD / ST	Load from data RAM; store result at a new address.
10	PUSH / POP	Push a value; clobber register; pop restores it.
11	Illegal opcode	Opcode 63 (undefined) routes to vector 5 handler.
12	IRQ0 + IRET	Hardware IRQ fires; handler runs; IRET returns to main.
13	Conditional exec	ADD.z, MOV.c, MOV.n both taken and suppressed.

7.3 Build System

The project ships with `build.sh`, a Bash script wrapping GHDL for simulation and generating a Vivado Tcl project script for synthesis. The following targets are available:

Table 7.2: build.sh targets

Target	Description
`analyse`	Analyse all VHDL sources into the GHDL work library.
`elaborate`	Elaborate the top-level entity.
`sim`	Full analysis + elaboration + simulation. Produces `wave.vcd`.
`clean`	Remove all GHDL build artefacts.
`vivado`	Write a Vivado Tcl project script for synthesis on the Basys 3.

7.3.1 Running Simulation

```
./build.sh sim
```

This runs all 13 test suites and prints `[PASS]` or `[FAIL]` for each check. A timeout of 200 μs is enforced; if the simulation does not complete within that time it is aborted with a `TIMEOUT` failure.

Waveforms are written to `wave.vcd` and can be inspected with GTKWave:

```
gtkwave wave.vcd
```

7.3.2 Synthesis

```
./build.sh vivado
vivado -mode batch -source vivado_project.tcl
```

The generated Tcl script creates a Vivado project targeting the `xc7a35tcpg236-1` (Basys 3), adds all source files, sets VHDL 2008 file type, applies the XDC constraints from `basys3.xdc`, sets the top entity to `top`, and launches synthesis.

7.4 Requirements

- **Simulation**: GHDL with VHDL-2008 support. GTKWave for waveform viewing.
- **Synthesis**: Vivado (any recent version targeting Artix-7). A Basys 3 board for hardware testing.
- **Assembler**: Python 3 (no external dependencies) for `as.py`.

Chapter 8

Assembler

8.1 Overview

The 4332 assembler is a two-pass Python script (`as.py`). Pass 1 tokenises the source, resolves label addresses, and computes instruction sizes. Pass 2 emits machine code using the resolved label table. It produces raw binary, ASCII hex, or VHDL `textio` format output.

8.2 Usage

```
python3 as.py [options] input.asm

Options:
  -o, --output FILE    Output file (default: a.out)
  -f, --format FORMAT  Output format: bin | hex | textio (default: bin)
```

The `textio` format produces one 16-bit word per line as a string of 16 `0/1` characters, directly readable in VHDL via `std.textio` with a `bit_vector(15 downto 0)` variable.

8.3 Register Syntax

Registers are written with a `%` prefix: `%0`–`%7`.

8.4 Mnemonics and Suffixes

Mnemonics are case-insensitive. Suffixes are separated from the base mnemonic by a dot (`.`) or slash (`/`). Width suffixes (`w`, `l`, `h`, `d`) apply to data instructions; condition suffixes (`z`, `c`, `n`) apply to all instructions. Both may be combined:

```
  add.l.z  %1, %2     ; Add low bytes, only if Z=1
  mov.z    %3, %0     ; Move to %3 if Z=1
  j.c      target     ; Jump if C=1
```

Width suffixes on `J` and `CALL` are not permitted (they encode addressing mode, not data width, and the assembler selects the mode automatically from the operand form).

8.5 Addressing Modes for J and CALL

The assembler selects the addressing mode based purely on the operand form:

```
j     target       ; AM_DIRECT: 2 words, abs address in extension word
j     %3           ; AM_INDIRECT: 1 word, target = %3
j     %3, 0x10     ; AM_INDIR_OFF: 2 words, target = %3 + extension word
```

8.6 LOADIMM

`LOADIMM` always assembles to two words: the opcode word (with the destination register in `RD`) followed by the 16-bit immediate value:

```
loadimm  %2, 0x1234    ; %2 <- 0x1234 (two instruction words)
loadimm  %0, label     ; %0 <- address of 'label'
```

Negative values and labels are both accepted. Immediates must fit in 16 bits.

8.7 Directives

Table 8.1: Assembler directives

Directive	Description
`.dw expr [, expr ...]`	Emit one or more raw 16-bit words.
`.db byte [, byte ...]`	Emit bytes packed two-per-word (must be even count).
`.ascii "string"`	Emit packed bytes from a string literal (no terminator).
`.asciz "string"`	Like `.ascii` but appends a null byte.
`.str "string"`	Alias for `.ascii`.
`.strz "string"`	Alias for `.asciz`.

String literals support the following escape sequences: `\n` (LF), `\r` (CR), `\t` (tab), `\0` (null), `\\` (backslash), `\"` (quote), `\xHH` (hex byte). Strings packed with `.ascii` or `.asciz` must have an even byte count (including any null terminator); the assembler raises an error if this is not the case.

8.8 Labels

A label is a valid identifier followed by a colon:

```
loop_start:
  dec    %0
  j.n    loop_start    ; branch back while N=1 (result < 0 never happens
                       ;   if %0 starts positive, this illustrates .n usage
```

Labels may refer to either code or data addresses; there is no distinction. Label names must consist of letters, digits, and underscores, must not start with a digit, and are case-sensitive. Forward references are supported (pass 1 resolves all labels before pass 2 emits code).

8.9 Number Literals

Numbers are written in decimal by default, hexadecimal with an `0x` prefix, or binary with an `0b` prefix.

8.10 Comments

Lines starting with `;` or `#` are comments. Inline comments may appear after any instruction using either character.

Appendix A

Instruction Set Reference

A.1 Instruction Listing by Group

Mnemonic	Opcode	Description
Arithmetic and Logic		
ADD	0x00	%0 ← %0 + %RS
SUB	0x01	%0 ← %0 − %RS
AND	0x02	%0 ← %0 AND %RS
OR	0x03	%0 ← %0 OR %RS
XOR	0x04	%0 ← %0 XOR %RS
NOT	0x05	%0 ← NOT %0
SHL	0x06	%0 ← %0 ≪ 1; C ← shifted-out bit
SHR	0x07	%0 ← %0 ≫ 1; C ← shifted-out bit
INC	0x08	%0 ← %0 + 1
DEC	0x09	%0 ← %0 − 1
CMP	0x0A	flags ← %0 − %RS; no writeback
MOV	0x0B	%RD ← %RS (explicit destination via RD_FROM_INST)
LOADIMM	0x0C	%RD ← next instruction word (2-word instruction)
PASSA	0x0D	%0 ← %RD (pass A-path through ALU)
PASSB	0x0E	%0 ← %RS (pass B-path through ALU)
NOP	0x0F	No operation
Memory		
LD	0x10	%RD ← mem[%RS] (word)
ST	0x11	mem[%RD] ← %RS (word)
LDB	0x12	%RD ← mem[%RS] (byte)
STB	0x13	mem[%RD] ← %RS (byte)
PUSH	0x14	RSP ← RSP − 1; mem[RSP] ← %RS
POP	0x15	%RD ← mem[RSP]; RSP ← RSP + 1
PEEK	0x16	%RD ← mem[RSP] (no SP change)
FLUSH	0x17	Pipeline flush hint (currently NOP)
Control Flow		

(continued)

Mnemonic	Opcode	Description
`J`	0x18	Jump unconditionally (3 addressing modes; see Section 2.2.2)
`J.z`	0x18+CE=01	Jump if Z=1
`J.c`	0x18+CE=10	Jump if C=1
`J.n`	0x18+CE=11	Jump if N=1
`CALL`	0x21	Push return address; jump (3 addressing modes)
`RET`	0x22	RPC ← pop()
`IRET`	0x23	RPC ← RIP; restore mode; GIE ← 1 (kernel only)
`INT`	0x24	Set pending bit for vector 4 (software interrupt)
`HLT`	0x25	Halt: PC stalls on this instruction forever
System (kernel mode only)		
`ENIRQ`	0x30	GIE ← 1
`DISIRQ`	0x31	GIE ← 0
`SETMODE`	0x32	RM ← `%RS`[1:0]
`GETMODE`	0x33	`%0` ← {RM_PREV, RM}
`WRITERIC`	0x34	RIC ← `%RS`
`READRIC`	0x35	`%0` ← RIC
`READRIV`	0x36	`%0` ← RIV[`%RS`]
`WRITERIV`	0x37	RIV[`%RS`] ← `%RD`
`HALT`	0x39	Alias for HLT

A.2 Instruction Descriptions

ADD — Add

Syntax: `add[.w|.l|.h|.d][.z|.c|.n] %rd, %rs`
Operation: `%0` ← `%0` + `%RS`
Flags: Z, C, N, V updated.
Notes: When `RD_FROM_INST` is not set, writeback always goes to `%0`. ADD with explicit `RD` field requires `MOV` to move the result. The accumulator-centric design is intentional: most tight loops do their work through `%0` and MOV out the result when done.

SUB — Subtract

Syntax: `sub[.w|.l|.h|.d][.z|.c|.n] %rd, %rs`
Operation: `%0` ← `%0` − `%RS`
Flags: Z, C (borrow), N, V updated.

CMP — Compare

Syntax: `cmp[.w|.l|.h|.d][.z|.c|.n] %rd, %rs`
Operation: flags ← `%0` − `%RS` (result discarded)
Flags: Z, C, N, V updated.
Notes: The `RD` field is ignored (no writeback). Use `CMP` before a conditional jump to test relationships between registers.

MOV — Move

Syntax: `mov[.w|.l|.h|.d][.z|.c|.n] %rd, %rs`
Operation: `%RD` ← `%RS`
Flags: Not updated (PASSB does not update flags).
Notes: MOV is the only standard ALU instruction that uses an explicit destination (`RD_FROM_INST` = 1 in its CW).

LOADIMM — Load Immediate

Syntax: `loadimm %rd, imm16`
Operation: `%RD` ← imm16
Size: 2 words.
Flags: Not updated.
Notes: The `DW` and `CE` fields in the opcode word are both ignored. The immediate is always the full 16-bit extension word. There is no short immediate form; use ALU instructions with the 4-bit `SB_IMM` source for small constants.

LD — Load Word

Syntax: `ld[.z|.c|.n] %rd, %rs`
Operation: `%RD` ← mem[`%RS`]
Size: 2 micro-steps.
Notes: Step 0 presents the address; step 1 latches the result from `ddata_r`. The address register is `%RS`; the destination is `%RD`.

ST — Store Word

Syntax: `st[.z|.c|.n] %rd, %rs`
Operation: mem[`%RD`] ← `%RS`
Notes: The *address* is in `%RD` and the *data* to write is in `%RS`. This is the opposite of the conventional load direction; it follows from the instruction encoding where `RD` is the "first operand" and `RS` is the "second operand."

PUSH — Push to Stack

Syntax: `push[.z|.c|.n] %rs`
Operation: RSP ← RSP − 1; mem[RSP] ← `%RS`
Notes: Pre-decrement: the new RSP value is the address written.

POP — Pop from Stack

Syntax: `pop[.z|.c|.n] %rd`
Operation: `%RD` ← mem[RSP]; RSP ← RSP + 1
Notes: Post-increment: the value at the current RSP is read, then RSP is incremented.

J — Jump

Syntax: `j[.z|.c|.n] target` or `j[.z|.c|.n] %rs` or `j[.z|.c|.n] %rs, offset`
Operation: RPC ← target (if condition met)
Notes: The assembler selects the addressing mode based on operand form (see Section 2.2.2). The `J` opcode subsumes all conditional and unconditional jumps; what were previously separate `JZ`, `JNZ`, `JC`, `JA`, etc. opcodes in the testbench are now encoded as `J` with the appropriate `CE` field.

Note: The testbench still uses separate `JZ`/`JNZ`/`JC`/`JA`/etc. opcode constants for historical reasons. These map to opcodes 25–32 and have their own CW entries with the appropriate `BRANCH_COND` setting.

CALL — Call Subroutine

Syntax: `call[.z|.c|.n] target` or `call[.z|.c|.n] %rs`
Operation: RSP ← RSP − 1; mem[RSP] ← RPC+1; RPC ← target
Notes: The return address pushed is the address of the instruction following `CALL` (or its extension word). Conditional calls are supported: if the condition is not met, neither the push nor the jump occurs.

WRITERIV / READRIV — Write / Read Interrupt Vector

Syntax: `writeriv %rd, %rs` / `readriv %rs`
Operation (WRITERIV): RIV[`%RS`] ← `%RD`
Operation (READRIV): `%0` ← RIV[`%RS`]
Notes: `%RS` holds the vector number (0–7). These are kernel-only instructions.

Appendix B

Assembler Language Reference

B.1 Source File Format

Source files are plain text with one statement per line. Lines are processed in order; the assembler is not whitespace-sensitive beyond token separation.

B.2 Comments

Single-line comments are introduced by `;` or `#`. Everything from the comment character to the end of the line is ignored.

```
  ; This is a comment
  add  %0, %1   # inline comment
```

B.3 Labels

A label definition is a valid identifier immediately followed by a colon, on its own line or before a statement. Labels are case-sensitive.

A valid identifier starts with a letter or underscore and contains only letters, digits, and underscores. Labels may not begin with a digit.

```
main:
  loadimm  %0, 42
loop_top:
  dec      %0
  j.n      loop_top
  hlt
```

B.4 Number Literals

Prefix	Base	Example
none	Decimal	`255`, `-1`
`0x`	Hexadecimal	`0xFF`, `0x1234`
`0b`	Binary	`0b1010`, `0b11110000`

B.5 String Escape Sequences

Sequence	Value	Description
`\n`	0x0A	Line feed
`\r`	0x0D	Carriage return
`\t`	0x09	Horizontal tab
`\0`	0x00	Null byte
`\\`	0x5C	Backslash
`\"`	0x22	Double quote
`\xHH`	0xHH	Arbitrary byte (two hex digits)

Appendix C

Control Word Layout

The 32-bit control word is laid out as follows. Every bit not listed is either reserved or part of a field shown below. All unset bits default to a safe/NOP value for their respective datapath.

Bits **Description**	**Field**	**Widt**
[31:28] ALU operation select. See Table C.2.	`ALU_OP`	4
[27:25] ALU B-input source. See Table C.3.	`SRC_B`	3
[24] Register file write enable.	`REG_WE`	1
[23] 1 = use `RD` from instruction word as destination; 0 = write to `%0`.	`RD_FROM_INST`	1
[22:20] Writeback data source. See Table C.4.	`WB_SRC`	3
[19] 1 = take effective width from `inst[3:2]`; 0 = use `WIDTH` field.	`WIDTH_FROM_INST`	1
[18:17] Width override when `WIDTH_FROM_INST` = 0.	`WIDTH`	2
[16] Assert `dbe` (data bus read enable).	`MEM_RD`	1
[15] Assert `dwe` (data bus write enable).	`MEM_WR`	1
[14] 1 = word access (both byte lanes); 0 = byte access (use dwmask).	`MEM_WIDTH`	1
[13:12] Memory address / write data source. See Table C.5.	`MEM_SRC`	2
[11:9] PC update source. See Table C.6.	`PC_SRC`	3
[8:5] Branch condition. See Table C.7.	`BRANCH_COND`	4
[4:2]	`SP_OP`	3

(continued)

Bits Description	Field	Widt
Stack pointer operation. See Table C.8.		
[1] 1 = instruction requires kernel mode; raises PRIV_FAULT if in user mode.	`PRIV_CHECK`	1
[0] 1 = this is the final micro-step; reset micro-step counter on next cycle.	`LAST_STEP`	1

Table C.2: ALU_OP encodings

Value	Name	Operation
0000	ALU_ADD	A + B
0001	ALU_SUB	A − B (also CMP)
0010	ALU_AND	A AND B
0011	ALU_OR	A OR B
0100	ALU_XOR	A XOR B
0101	ALU_NOT	NOT A
0110	ALU_SHL	A $\ll$ 1 (carry out = A[MSB])
0111	ALU_SHR	A $\gg$ 1 (carry out = A[0])
1000	ALU_INC	A + 1
1001	ALU_DEC	A − 1
1010	ALU_CMP	A − B (flags only, same as SUB)
1011	ALU_PASSA	A (pass through, flags not updated)
1100	ALU_PASSB	B (pass through, flags not updated)
1101	ALU_NOP	Zero (no flags updated)

Table C.3: SRC_B encodings

Value	Name	Source
000	SB_RS	Register file [`%RS`]
001	SB_IMM	4-bit zero-extended immediate from `inst[3:0]`
010	SB_MEM	Data bus read data (`ddata_r`)
011	SB_SP	Stack pointer (`RSP`)
100	SB_PC	Program counter (`RPC`)

Table C.4: WB_SRC encodings

Value	Name	Source
000	WB_ALU	ALU result
001	WB_MEM	Data bus read data (`ddata_r`)
010	WB_IMM	4-bit immediate (zero-extended)
011	WB_PC1	RPC + 1 (next instruction address)
100	WB_SPEC	Special register (decoded from opcode: RM, RIC, or RIV)
101	WB_IMM_WORD	Captured 16-bit immediate word (for LOADIMM step 1)

Table C.5: MEM_SRC encodings

Value	Name	Memory address / write data source
00	MS_RS	Register file [`%RS`]
01	MS_SP	Stack pointer (`RSP`)
10	MS_PC	RPC + 1 (for CALL: saves the return address)
11	MS_IMM	4-bit immediate (zero-extended)

Table C.6: PC_SRC encodings

Value	Name	Next-PC source
000	PC_SEQ	Sequential: RPC already incremented by FETCH
001	PC_JUMP	Jump: target determined by addressing mode
010	PC_STALL	Stall: RPC ← `de_pc` (hold at current instruction, HLT)
011	PC_INT	Interrupt: RPC ← RIV[vec]
100	PC_RET	Return: RPC ← `ddata_r` (popped return address)
101	PC_IRET	Interrupt return: RPC ← RIP; restore mode; GIE ← 1

Table C.7: BRANCH_COND encodings

Value	Name	Condition
0000	BR_NEVER	Never taken (used to prevent jump when PC_JUMP is set but branch is structural only)
0001	BR_ALWAYS	Always taken
0010	BR_Z	Z = 1
0011	BR_NZ	Z = 0
0100	BR_C	C = 1
0101	BR_NC	C = 0
0110	BR_A	C = 0 AND Z = 0 (above, unsigned)
0111	BR_B	C = 1 (below, unsigned; same as BR_C)
1000	BR_AE	C = 0 (above or equal)
1001	BR_BE	C = 1 OR Z = 1 (below or equal)

Table C.8: SP_OP encodings

Value	**Name**	**Stack pointer operation**
000	SP_NONE	No change
001	SP_PUSH	RSP $\leftarrow$ RSP $-$ 1 (pre-decrement before write)
010	SP_POP	RSP $\leftarrow$ RSP + 1 (post-increment after read)
011	SP_INC	RSP $\leftarrow$ RSP + 1 (standalone increment)
100	SP_DEC	RSP $\leftarrow$ RSP $-$ 1 (standalone decrement)

Appendix D

Register File

Table D.1: General-purpose registers

Register	Reset Value	Notes
`%0`	`0x0000`	Implicit ALU accumulator / result register
`%1`	`0x0000`	General purpose / argument register 1
`%2`	`0x0000`	General purpose / argument register 2
`%3`	`0x0000`	General purpose / argument register 3
`%4`	`0x0000`	General purpose
`%5`	`0x0000`	General purpose
`%6`	`0x0000`	General purpose
`%7`	`0x0000`	General purpose

Table D.2: Special-purpose registers

Register	Reset Value	Description
`RSP`	`0xBFFE`	Stack pointer (top of RAM bank 0)
`RPC`	`0x0000`	Program counter
`RIP`	`0x0000`	Interrupt return pointer (saved by hardware on interrupt entry)
`XR0`	`0x0000`	Exception context: faulting PC
`XR1`	`0x0000`	Exception context: faulting vector number
`XR2`	`0x0000`	Exception context: faulting instruction word
`XR3`	`0x0000`	Exception context: faulting memory address
`RM`	`0x0001`	Mode register: [0]=current mode, [1]=previous mode. 1=kernel, 0=user
`RIC`	`0x0000`	Interrupt controller: [15:8]=pending, [7:1]=enable[7:1], [0]=GIE
`RIV[0..7]`	`0x0000`	Interrupt vector table entries